

Optimal Market Making for Cryptocurrency Options

The Avellaneda–Stoikov Framework and its Extension to Options

North Carolina State University · Financial Mathematics

Summer 2026

NC STATE UNIVERSITY

Optimal Market Making for Cryptocurrency Options

The Avellaneda–Stoikov Framework and its Extension to Options

A 10-Week Summer Research Program

North Carolina State University — Department of Mathematics

Industry Mentor: Dendi Suhubdy — *Bitwyre*, San Francisco

Guest Practitioner: Fenni Kang — Chief Strategy Officer, *Coincall*

Edition: 2026

1. Course Overview

This ten-week summer research program prepares students in the North Carolina State University Financial Mathematics graduate program to design, implement, and evaluate optimal market making strategies for cryptocurrency options. The program is organized around the **Avellaneda–Stoikov framework**, the foundational stochastic-control approach to optimal liquidity provision. The central pedagogical question is how to extend Avellaneda–Stoikov, which was originally formulated for a single linear instrument, to non-linear

derivatives whose inventory risk is multi-dimensional in the Greeks. A dedicated advanced week treats extensions of Avellaneda–Stoikov to the multi-asset, adverse-selection, discrete-inventory, signal-driven, and robust settings.

Week 1 was an **introduction week**, delivered last week: it covered the program goals, the development environment, the Coincall datasets, and the logistics described in Sections 1.5–1.7. The nine technical weeks that follow (Weeks 2–10) are the substance of this syllabus.

The program is mentored by Dendi Suhubdy and includes two guest sessions from Fenni Kang, Chief Strategy Officer at Coincall, on **taker strategies** (Week 9) and on the **corporate usage of options and structured products** (Week 10). Students will work with live and historical Coincall data throughout the program and will produce a research-grade final report and code repository suitable for further academic or industry use.

1.1 Implementation Languages

The program uses a deliberate two-language split, and students should understand the rationale:

- **Problem sets are written in Python**, using **PyTorch** as the numerical engine. PyTorch is chosen so that *differentiation is handled implicitly by autograd*: students compute option Greeks, calibrate surfaces, and fit intensities by writing the forward computation only and letting `.backward()` produce the sensitivities. We deliberately do **not** ask students to hand-derive adjoint algorithmic differentiation; the goal is correct, readable research code, not AD plumbing.
- **Fast-computation methods are written in C++**. The latency-critical numerical kernels — characteristic-function pricers, implied-volatility inversion, and full-surface revaluation — are implemented in C++ for speed, then exposed to Python through a **pybind11** binding so that the same Python research code can call them. Week 7 teaches **pybind11** explicitly: how to write a binding module, build it with CMake, manage the GIL, and pass NumPy/PyTorch tensors across the boundary without copying.

The result is a realistic hybrid that mirrors industry practice: Python for research and orchestration, C++ for the hot path, and a clean binder between them.

1.2 Learning Objectives

Upon successful completion of this program, students will be able to:

1. Formulate a market making problem as a continuous-time stochastic optimal control problem and understand the associated Hamilton–Jacobi–Bellman (HJB) equation.
2. Solve the HJB system for the Avellaneda–Stoikov model in closed form and implement the resulting quoting policy as a Python/PyTorch engine validated against the analytical solution.
3. Calibrate parametric volatility surfaces (SVI, SABR) to live cryptocurrency options data in PyTorch and assess their stability across time.
4. Implement latency-critical pricing, implied-volatility, and Greek routines in C++ and call them from Python through a `pybind11` binding.
5. Extend the Avellaneda–Stoikov framework to a multi-Greek inventory penalty following El Aoud and Abergel, and reduce the resulting HJB system to a numerically tractable form.
6. Integrate a dynamic hedging policy with the quoting engine and analyze the resulting P&L attribution.
7. Build and validate an event-driven backtest that consumes real Coin-call order-book data and produces interpretable P&L, risk, and Greek-exposure diagnostics.
8. Situate the maker (market-making) perspective against **taker strategies**, and understand how options are used by corporates through **structured products** (collars, accumulators/decumulators, coupon products).
9. Implement at least one advanced extension of Avellaneda–Stoikov and communicate the results in writing and orally at a level suitable for industry quantitative-research interviews.

1.3 Prerequisites

Students are expected to have completed or be concurrently enrolled in graduate coursework covering: stochastic calculus (Itô integration, Itô’s lemma, Girsanov’s theorem), continuous-time stochastic processes (Brownian motion, Poisson processes, jump-diffusions), Black–Scholes–Merton option pricing, partial differential equations at the level of Evans, and probability theory at the level of Durrett.

Programming maturity in **Python** is required, including comfort with NumPy and the basics of **PyTorch** (tensors, autograd). Familiarity with **modern C++ (C++17)** and **CMake** is advantageous for the fast-computation week but is taught from a working scaffold; no prior `pybind11` experience is assumed.

1.4 Structure

The program runs for ten weeks. Week 1 was the introduction week (delivered last week). Each of the nine technical weeks (Weeks 2–10) comprises two ninety-minute lectures, one ninety-minute recitation, and approximately fifteen hours of independent reading, programming, and benchmarking. **Weeks 5, 7, and**

9 are milestone weeks at which students deliver a working software component built on the preceding weeks' material. The final week (Week 10) surveys advanced Avellaneda–Stoikov extensions and is devoted to a chosen extension topic and the preparation and presentation of a research-grade final report.

The schedule is built around the Avellaneda–Stoikov framework as the unifying worked example: it is introduced in Week 3, placed within the general continuous-time control template in Week 4, implemented as a production engine in Week 5, and extended to a multi-Greek options setting in Week 8. Extensions of Avellaneda–Stoikov are consolidated into the dedicated advanced week (Week 10) rather than being interleaved throughout.

1.5 Assessment

Final grades are determined as follows. Weekly problem sets account for thirty percent of the grade. The three milestone deliverables in Weeks 5, 7, and 9 jointly account for thirty percent. The final report and presentation account for thirty percent. Active participation in lecture, recitation, and code review accounts for the remaining ten percent.

The weekly problem sets are distributed as a separate **Problem Sets** booklet; fully worked solutions are distributed (to graders, and to students after each deadline) as a separate **Solutions** booklet.

1.6 Mentorship Model

Because the industry mentor contributes on a volunteer basis alongside a full-time role, mentorship is primarily **asynchronous** and carries **no fixed weekly time commitment**. The mentor is reachable by **email and Telegram**, and aims to respond within a few business days. This channel is intended for clarification of difficult material, code review of milestone deliverables, and discussion of research directions for the final week; students should send specific, well-prepared questions (including a minimal reproducible code snippet where relevant).

When a question genuinely needs a live conversation, **bespoke one-on-one sessions are arranged ad hoc**, subject to the mentor's availability, rather than on a standing weekly schedule. Day-to-day support — recitation, office hours, and first-pass grading — is provided by the program tutors, who escalate to the mentor as needed.

1.7 Guest Practitioner Sessions

Fenni Kang (Chief Strategy Officer, Coincall) contributes two guest sessions, integrated as additional lectures within the regular schedule:

- **Week 9 — Taker Strategies.** A practitioner’s view of liquidity *consumption*: when and why takers cross the spread, the signals that drive aggressive flow, and how taker behaviour shapes the order flow that the market maker must price.
- **Week 10 — Corporate Usage of Options and Structured Products.** How corporates and treasuries use options: collars to fix a price band, accumulators and decumulators for staged accumulation/liquidation, and coupon (yield-enhancement) products. The session connects the market-maker’s book to the structured-product flow that generates it.

2. Program Milestones at a Glance

The program is organized around three milestone deliverables. Each milestone integrates the material from the preceding weeks and produces a software artifact that is incorporated into the final research repository.

Week	Title	Deliverable
5	Linear-asset Avellaneda–Stoikov engine	Python/PyTorch implementation of the Avellaneda–Stoikov model on a single linear asset, validated against simulated dynamics and benchmarked on one month of Coincall BTC-PERP order-book data.
7	Fast pricing and Greeks library	C++ pricing/implied-vol/Greeks library for European options under Black–Scholes and Heston dynamics, exposed to Python via <code>pybind11</code> , with PyTorch reference Greeks. Latency target: full Coincall BTC surface revaluation in under 5 milliseconds.
9	Full options market-making backtest	Event-driven backtest of the multi-Greek market-making engine on three months of Coincall BTC and ETH options data, with full P&L attribution, risk metrics, and Greek-exposure time series.

3. Week-by-Week Schedule

Each of the following sections gives the learning objectives, lecture outlines, recitation activity, required reading, and (where applicable) milestone deliverable for one week of the program. Detailed problem-set questions live in the separate Problem Sets booklet; each week below lists the problem set’s scope and its implementation language.

Week 1 — Introduction (Delivered)

Theme: orientation, environment, and data. This week was delivered last week and is recorded here for completeness.

Week 1 introduced the program’s goals and arc, the maker-versus-taker framing, and the development environment. Students set up the Python + PyTorch toolchain, a C++17 toolchain with CMake, and a working `pybind11` “hello world” binding; received credentials to the Coincall data store; and signed the data-use agreement. There was no graded problem set for Week 1.

Week 2 — Foundations of Market Microstructure

Theme: from limit order books to the economic role of the market maker.

Learning Objectives

- Reconstruct a limit order book from raw exchange feeds and compute standard liquidity metrics.
- Articulate the three classical components of the bid-ask spread and identify them in real data.
- Explain the role of the market maker as a continuously quoting liquidity provider, and contrast it with directional and taker trading.

Lecture 1 — Limit Order Book Mechanics. Order types (market, limit, IOC, post-only, hidden); price-time priority and the matching algorithm; L1/L2/L3 order-book views; tick size and lot size; and the Coincall exchange architecture (contract specifications, fee structure, market maker tiers).

Lecture 2 — Spread Decomposition and the Role of the Market Maker. The bid-ask spread decomposed into adverse selection, inventory, and

order-processing components, via the classical models of Roll (1984), Glosten–Milgrom (1985), and Kyle (1985); realized spread, effective spread, and price impact; why crypto microstructure differs from equities.

Recitation. Reconstruct the order book from a Coincall tick-level recording in Python: replay one hour of BTC-PERP events and produce a time series of mid-prices, spreads, and book depth. Output: a Python class that maintains live order-book state from raw events.

Required Reading. Cartea, Jaikumar, and Penalva (2015), Ch. 1–3; Roll (1984); Glosten and Milgrom (1985).

Problem Set 2 (Python). Order-book reconstruction, the Roll estimator, the Glosten–Milgrom quote-setter (belief update simulated in PyTorch), and a realized-spread / price-impact decomposition on Coincall data. See the Problem Sets booklet.

Week 3 — The Avellaneda–Stoikov Model

Theme: a fully worked example of optimal market making through stochastic control.

Learning Objectives

- Set up the Avellaneda–Stoikov problem as a continuous-time stochastic control problem with exponential utility.
- Understand the associated HJB equation and its reduction to a system of ODEs through the standard ansatz.
- Interpret the closed-form reservation price and optimal half-spread economically.

Lecture 1 — Setup and Dynamics. Mid-price as arithmetic Brownian motion; Poisson order flow with intensity $\lambda(\delta) = A \cdot \exp(-\kappa\delta)$; wealth and inventory dynamics; exponential (CARA) utility and why it makes the problem tractable; the terminal liquidation penalty.

Lecture 2 — HJB Solution. The dynamic programming principle and the HJB equation; the separation ansatz $V(t, x, S, q) = -\exp(-\gamma(x + qS)) \cdot \theta(t, q)$; reduction to ODEs in θ ; the reservation price $r = S - q\gamma\sigma^2(T - t)$; and the optimal half-spread and its decomposition into risk-aversion and order-flow components.

Recitation. Walk through the derivation together, then implement the closed-form quotes in PyTorch and visualize the reservation price and spread as functions of inventory.

Required Reading. Avellaneda and Stoikov (2008); Cartea, Jaikumar, and Penalva (2015), Ch. 10.

Problem Set 3 (Python/PyTorch). A PyTorch ODE solver for $\theta(t,q)$ validated against the closed-form quotes; a reusable quote engine; a one-day simulation comparing optimal vs. symmetric quoting; a γ sensitivity sweep; and a price-impact experiment. See the Problem Sets booklet.

Week 4 — Stochastic Optimal Control in Continuous Time

Theme: the general theory underlying the previous week's worked example.

Learning Objectives

- State and apply the dynamic programming principle in a general continuous-time setting.
- Understand the HJB equation for a generic control problem and the role of the verification theorem.
- Recognize the role of CARA and CRRA utilities in producing tractable problems.

Lecture 1 — Dynamic Programming and the HJB Equation. Controlled diffusions and admissible controls; the dynamic programming principle; the infinitesimal generator; derivation of the HJB equation; boundary and terminal conditions.

Lecture 2 — Verification Theorems and Tractable Utilities. The verification theorem and smooth-fit conditions; viscosity solutions as the setting for non-smooth value functions; CARA, CRRA, and HARA families; why CARA decouples the value function from wealth in market making.

Recitation. Worked solutions to Merton's portfolio problem under CARA and CRRA, and an optimal-liquidation problem with linear impact, identifying the recurring pattern: HJB $\rightarrow$ ansatz $\rightarrow$ ODE $\rightarrow$ solution.

Required Reading. Pham (2009), Ch. 3–4; Cartea, Jaikumar, and Penalva (2015), Ch. 6.

Problem Set 4 (Python/PyTorch). PyTorch solvers for the Merton problem under CRRA and CARA; an Almgren–Chriss optimal-execution solver; an HJB-residual verification harness; and an obstacle-problem (variational-inequality) solver illustrating where viscosity solutions are needed. See the Problem Sets booklet.

Week 5 — The Avellaneda–Stoikov Engine: Implementation, Calibration, and Backtest

*Theme: turning the closed-form solution into a calibrated, backtested quoting engine. **Milestone 1.***

Learning Objectives

- Implement the Avellaneda–Stoikov quotes as a clean, tested Python/PyTorch engine validated against the analytical solution.
- Calibrate the order-flow intensity functions from historical Coincall fill data by maximum likelihood.
- Build an event-driven backtest harness and benchmark the engine on one month of Coincall BTC-PERP data.

Lecture 1 — From Closed Form to Code. Engine architecture; state management (inventory, cash, mid-price estimation from the L2 book); mapping continuous-time quantities to discrete event arrivals; calibrating $\lambda(\delta) = A \cdot \exp(-\kappa\delta)$ from fill data; numerical pitfalls (units, cash-process sign conventions, inventory indexing).

Lecture 2 — Calibration, Simulation, and Validation. Maximum-likelihood calibration of A and κ with goodness-of-fit; the backtest harness (replay engine, fill simulator, P&L attribution); validating the P&L distribution against theory; reproducibility under a fixed seed.

Recitation. Code review of student engines in progress; each student walks through one simulated trading day; group resolution of common bugs; walk-through of the Milestone 1 acceptance criteria.

Required Reading. Avellaneda and Stoikov (2008) (re-read with implementation in mind); Guéant (2016), Ch. 4 (single-asset market making, engineering background).

Problem Set 5 (Python/PyTorch). Constitutes Milestone 1: an optimized quote path, ML intensity calibration, reproduction of the paper’s inventory/P&L behaviour, a one-month backtest vs. a symmetric baseline, and a `pytest` suite with reproducible seeds. See the Problem Sets booklet.

Milestone Deliverable — Milestone 1: Linear-Asset Avellaneda–Stoikov Engine. A Python/PyTorch package implementing the Avellaneda–Stoikov model on a single linear asset, with calibration, simulation, and backtesting infrastructure, and a reproducible one-month backtest on Coincall BTC-PERP data documented in a short technical report.

Datasets: `coincall_btc_perp_l2_2025Q4.parquet`, `coincall_btc_perp_trades_2025Q4.parquet`, `coincall_intensity_calibration_set.parquet`.

Acceptance criteria: quote computation vectorized over inventory in PyTorch; intensity calibration with χ^2 goodness-of-fit; backtest P&L within $\pm 50\%$ of the mentor's reference; reproducible under a fixed seed; `pytest` coverage of the core quote computation; report ≤ 10 pages.

Week 6 — Options Pricing, the Volatility Surface, and Calibration

Theme: options theory and the practical construction of a volatility surface.

Learning Objectives

- Recall the Greeks of European options, including the second-order Greeks vanna and volga.
- State the no-static-arbitrage constraints on the volatility surface and verify them on parametric surfaces.
- Calibrate SVI and SABR parameterizations to live Coincall options data with attention to stability.

Lecture 1 — Greeks and the Volatility Surface. Black–Scholes and the Greek vector; vanna, volga, and volatility risk; the implied-volatility smile and term structure; calendar and butterfly arbitrage; features of the crypto options surface (sparse strikes, wide wings, quote staleness).

Lecture 2 — Parameterization and Calibration. Raw SVI and the arbitrage-free SVI/eSSVI formulation; SVI calibration by nonlinear least squares; the SABR formula and the Hagan expansion; time-stability of calibrated parameters; diagnostic plots.

Recitation. Live calibration in PyTorch: load a Coincall chain snapshot, fit SVI and SABR (using autograd for the Jacobians), and produce the standard diagnostics.

Required Reading. Gatheral (2006), Ch. 1–5; Gatheral and Jacquier (2014); Hagan et al. (2002); Hull (2021), Ch. 15, 19–21.

Problem Set 6 (Python/PyTorch). Black–Scholes price and Greeks with PyTorch autograd (including vanna/volga); SVI with arbitrage-free constraint checks; SVI and SABR calibration to a Coincall snapshot; and a one-month daily-calibration stability study with smoothing. See the Problem Sets booklet.

Week 7 — Fast Computation of Prices, Greeks, Implied Volatility, and Surfaces

*Theme: the latency budget of an options market maker, and the C++ numerical methods that meet it. **Milestone 2.***

Learning Objectives

- Implement and benchmark characteristic-function pricing methods (Carr–Madan FFT and Fang–Oosterlee COS) in C++.
- Implement robust, fast implied-volatility inversion and full-surface revaluation in C++.
- Expose the C++ kernels to Python with `pybind11`, and validate Greeks against PyTorch autograd.

Lecture 1 — Characteristic-Function Pricing and Implied Volatility in C++. The characteristic function under Black–Scholes and Heston; the Carr–Madan FFT method; the Fang–Oosterlee COS method and its truncation interval; robust implied-vol inversion (Jäckel-style rational starts plus a safeguarded Newton/Halley step); vectorizing a whole surface.

Lecture 2 — Binding C++ to Python with `pybind11`. This is the binder lecture. How a `pybind11` module is structured and built with CMake; passing NumPy/PyTorch tensors across the boundary without copying (buffer protocol, `py::array_t`); releasing the GIL around the C++ hot loop; and how to keep a single source of truth for Greeks by validating the C++ results against PyTorch autograd. We rely on PyTorch for *reference* Greeks rather than hand-rolling adjoint AD in C++ — autograd is the oracle, C++ is the speed.

Recitation. Profile a naive Python pricer, move the kernel to C++, bind it with `pybind11`, and re-benchmark — targeting at least an order of magnitude speedup and a sub-5ms full-surface revaluation.

Required Reading. Carr and Madan (1999); Fang and Oosterlee (2008); Jäckel (2015), *Let's Be Rational*; the `pybind11` documentation (overview, NumPy, and GIL sections).

Problem Set 7 (C++ with `pybind11`, called from Python). Constitutes Milestone 2: a C++ COS/Carr–Madan pricer, a robust C++ implied-vol solver, a vectorized surface revaluation, all bound to Python via `pybind11`, with Greeks validated against PyTorch autograd and latency reported as a distribution. See the Problem Sets booklet.

Milestone Deliverable — Milestone 2: Fast Pricing and Greeks Library. A C++ library that prices European options under Black–Scholes and Heston via characteristic-function methods, inverts implied volatility robustly, computes the full Greek vector,

and revalues the entire calibrated Coincall surface within a strict latency budget — all exposed to Python through a `pybind11` module. Greeks are validated against an independent PyTorch-autograd reference.

Datasets: `coincall_btc_options_chain_snapshots_2025Q4.parquet`, `coincall_calibrated_svi_surfaces_2025Q4.parquet`, `coincall_reference_prices_2025Q4.parquet`

Acceptance criteria: full BTC surface (300–500 contracts) revalued with full Greek vector in under 5 ms on one CPU core; max relative pricing error $1e-4$ vs. reference; Greeks within $1e-4$ of PyTorch autograd; clean `pybind11` API importable from Python; tests over all pricers, the implied-vol solver, and the surface adapter; latency reported as a distribution, not just a mean.

Week 8 — Options Market Making Theory and Dynamic Hedging

Theme: extending Avellaneda–Stoikov to a multi-Greek inventory penalty, and integrating a dynamic hedging policy.

Learning Objectives

- Set up the El Aoud–Abergel framework, in which inventory is a vector of Greeks.
- Understand the multi-Greek HJB system and the inventory penalty as a quadratic form.
- Understand delta and gamma hedging and quantify the P&L attribution of a hedged options book.

Lecture 1 — Multi-Dimensional Inventory and the Multi-Greek HJB.

The inventory vector $q = (\Delta, \Gamma, \nu, \dots)$ and its dynamics under quoting; the quadratic inventory penalty and its economic interpretation; the multi-Greek HJB and separation ansatz; reduction to a problem on the inventory vector; verification that it reduces to Avellaneda–Stoikov in the one-dimensional case.

Lecture 2 — Dynamic Hedging and Integration with Quoting. Continuous and discrete delta hedging; the gamma–theta–realized-variance identity; gamma scalping; vega hedging and skew risk; external vs. integrated hedging architectures; perpetual futures as the delta-hedging instrument and funding-rate considerations on Coincall.

Recitation. Multi-Greek HJB walk-through, then a PyTorch hedged-book simulation comparing hedge policies and P&L distributions.

Required Reading. El Aoud and Abergel (2015); Taleb (1997), selected chapters; Cartea, Jaikumar, and Penalva (2015), Ch. 11.

Problem Set 8 (Python/PyTorch). Multi-Greek inventory aggregation and dynamics; a PyTorch solver for the El Aoud–Abergel reduced problem verified to collapse to Avellaneda–Stoikov in 1-D; penalty-matrix calibration from Coincall Greek covariances; numerical verification of the gamma–theta–variance identity; and a threshold delta-hedger integrated with the Week 5 engine. See the Problem Sets booklet.

Week 9 — Backtesting, Risk Analysis, and Taker Strategies

*Theme: assembling the full system and validating it on historical data, with a practitioner’s view of taker flow. **Milestone 3.***

Learning Objectives

- Implement an event-driven backtest harness suitable for options market making.
- Produce a full P&L attribution (spread, inventory, gamma, vega, hedging).
- Analyze risk metrics and identify regimes of out- and under-performance.
- Understand taker strategies and how aggressive flow shapes the maker’s problem.

Lecture 1 — Backtesting Architecture and Risk Metrics. Event-driven simulation (clock, event queue, deterministic replay); the fill simulator; state tracking (cash, inventory by contract, aggregated Greeks, hedge position); pitfalls (look-ahead bias, unrealistic fills, expiry handling); Sharpe/Sortino/Calmar, drawdown, and P&L attribution; the deflated Sharpe ratio and block-bootstrap confidence intervals.

Lecture 2 (Guest — Fenni Kang, Coincall) — Taker Strategies. A practitioner’s view of liquidity consumption: when and why takers cross the spread; the signals that drive aggressive flow (momentum, liquidation cascades, news); execution styles (sweep, TWAP/VWAP, iceberg-taking); and what taker behaviour implies for the toxicity of the order flow a market maker faces.

Recitation. Code review of the full backtest; each student presents one regime and walks through the P&L attribution.

Required Reading. Cartea, Jaikumar, and Penalva (2015), Ch. 10–11 (re-read); Bailey and López de Prado (2014); López de Prado (2018), Ch. 11–13.

Problem Set 9 (Python/PyTorch). Constitutes Milestone 3: an event-driven backtest with deterministic replay, a calibrated/validated fill simulator, a full multi-Greek backtest with risk metrics and Greek time series, the P&L attribution decomposition, and a worst-drawdown forensic with a taker-flow analysis informed by the guest lecture. See the Problem Sets booklet.

Milestone Deliverable — Milestone 3: Full Options Market-Making Backtest. A complete backtest of the multi-Greek engine on three months of Coincall BTC and ETH options data, with full P&L attribution, risk metrics, and Greek-exposure time series, reproducible from a configuration file and documented in a technical report (≤ 20 pages) that includes a drawdown forensic.

Datasets: `coincall_btc_options_12_2025Q4.parquet`, `coincall_btc_options_trades_2025Q4.parquet`, `coincall_eth_options_12_2025Q4.parquet`, `coincall_eth_options_trades_2025Q4.parquet`, `coincall_btc_eth_perp_12_2025Q4.parquet`, `coincall_funding_rates_2025Q4.parquet`.

Acceptance criteria: reproducible from config; attribution sums to total P&L within 1 bp; Sharpe reported with block-bootstrap CI; Greek exposures bounded within configured limits; report includes a forensic of at least one drawdown event.

Week 10 — Advanced Avellaneda–Stoikov, Structured Products, and Final Presentations

Theme: advanced extensions of Avellaneda–Stoikov, the corporate structured-products view, one chosen extension, and the communication of results.

Learning Objectives

- Survey advanced extensions of Avellaneda–Stoikov (multi-asset, adverse selection, discrete inventory, signal-driven, robust) and implement one.
- Understand how corporates use options through structured products.
- Write and present a research-grade report.

Lecture 1 — Advanced Avellaneda–Stoikov Extensions. Multi-asset market making across BTC and ETH options under a correlated covariance matrix; adverse-selection-aware quoting and skew trading from calibrated SVI parameters; signal-driven quoting (machine-learned order-flow prediction); discrete-inventory formulations; and robust optimization with a worst-case inventory penalty over a confidence set. Students choose one extension by end of day.

Lecture 2 (Guest — Fenni Kang, Coincall) — Corporate Usage of Options and Structured Products. How corporates and treasuries use options: **collars** to fix a price band on a holding; **accumulators** / **decumulators** for staged accumulation or liquidation; and **coupon** / **yield-enhancement products**. The session links these structured-product flows to the market-maker’s book — how a desk warehouses and hedges the resulting Greek exposure — and closes with guidance on research-report structure and the oral presentation.

Recitation. Final dry-run presentations to the cohort, with feedback focused on the questions an industry audience would ask.

Required Reading. Guéant (2016), the chapter most relevant to the chosen extension; El Aoud and Abergel (2015) (re-read with the extension in mind); one additional paper relevant to the chosen extension; for the guest session, a short structured-products primer distributed in advance.

Problem Set 10 (Python/PyTorch). A one-page extension proposal; a PyTorch implementation of the chosen advanced Avellaneda–Stoikov extension on top of the milestone infrastructure; a quantitative comparison against the Week 9 baseline; the final report (≤ 30 pages); and the final twenty-minute presentation. See the Problem Sets booklet.

4. Comprehensive Reference List

4.1 Primary References

Avellaneda, M., and Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance*, 8(3), 217–224.

El Aoud, S., and Abergel, F. (2015). A stochastic control approach to option market making. *Market Microstructure and Liquidity*, 1(1), 1550006.

Guéant, O. (2016). *The Financial Mathematics of Market Liquidity: From Optimal Execution to Market Making*. Chapman & Hall / CRC.

Cartea, Á., Jaikumar, S., and Penalva, J. (2015). *Algorithmic and High-Frequency Trading*. Cambridge University Press.

4.2 Volatility Surface and Options Pricing

Gatheral, J. (2006). *The Volatility Surface: A Practitioner’s Guide*. Wiley Finance.

Gatheral, J., and Jacquier, A. (2014). Arbitrage-free SVI volatility surfaces. *Quantitative Finance*, 14(1), 59–71.

Hagan, P. S., Kumar, D., Lesniewski, A. S., and Woodward, D. E. (2002). Managing smile risk. *Wilmott Magazine*, September, 84–108.

Jäckel, P. (2015). Let’s be rational. *Wilmott Magazine*.

Hull, J. C. (2021). *Options, Futures, and Other Derivatives* (11th ed.). Pearson.

Taleb, N. N. (1997). *Dynamic Hedging: Managing Vanilla and Exotic Options*. Wiley.

4.3 Fast Computation Methods

Carr, P., and Madan, D. (1999). Option valuation using the fast Fourier transform. *Journal of Computational Finance*, 2(4), 61–73.

Fang, F., and Oosterlee, C. W. (2008). A novel pricing method for European options based on Fourier-cosine series expansions. *SIAM Journal on Scientific Computing*, 31(2), 826–848.

Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. Springer.

4.4 Stochastic Optimal Control

Pham, H. (2009). *Continuous-time Stochastic Control and Optimization with Financial Applications*. Springer.

Fleming, W. H., and Soner, H. M. (2006). *Controlled Markov Processes and Viscosity Solutions* (2nd ed.). Springer.

Øksendal, B. (2003). *Stochastic Differential Equations* (6th ed.). Springer.

4.5 Market Microstructure

Roll, R. (1984). A simple implicit measure of the effective bid-ask spread. *Journal of Finance*, 39(4), 1127–1139.

Glosten, L. R., and Milgrom, P. R. (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1), 71–100.

Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica*, 53(6), 1315–1335.

4.6 Software and Tooling

Jakob, W., Rhineland, J., and Moldovan, D. (2017). *pybind11 — Seamless operability between C++11 and Python*. <https://github.com/pybind/pybind11>

Paszke, A., et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS*.

4.7 Supplementary

Bailey, D. H., and López de Prado, M. (2014). The deflated Sharpe ratio. *Journal of Portfolio Management*, 40(5), 94–107.

López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.

Almgren, R., and Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5–40.

Appendix A. Coincall Dataset Catalog

All datasets are delivered as Parquet files on a secure object store under a non-disclosure agreement. The naming convention is `coincall_{instrument}_{data_type}_{period}.parquet`.

- **Order book data** — L2 snapshots (top ten levels per side), at 1-second resolution for options and 100 ms for perpetual futures, each row carrying bid/ask prices and sizes per level, a strictly increasing sequence number, and an exchange timestamp. L2 is sufficient for both intensity calibration and the simulation environment.
- **Trade data** — the full trade tape (contract id, price, size, aggressor side, trade id); options trades additionally carry the mark implied volatility at trade time, used for surface calibration in Week 6 and the fill simulator in Week 9.
- **Surface data** — calibrated SVI parameters at five-minute frequency, used as a baseline for the fast revaluation pipeline in Week 7.
- **Auxiliary data** — hourly perpetual-futures funding rates (for hedging cost accounting in Week 9) and a static snapshot of contract specifications.

Students receive credentials to a secure S3-compatible endpoint at the start of the program. Data may be downloaded locally for analysis but may not be redistributed. Coincall retains the right to update the dataset catalog during the program with at least one week of notice.